



NUST CHIP DESIGN CENTRE

RISC-V Architecture

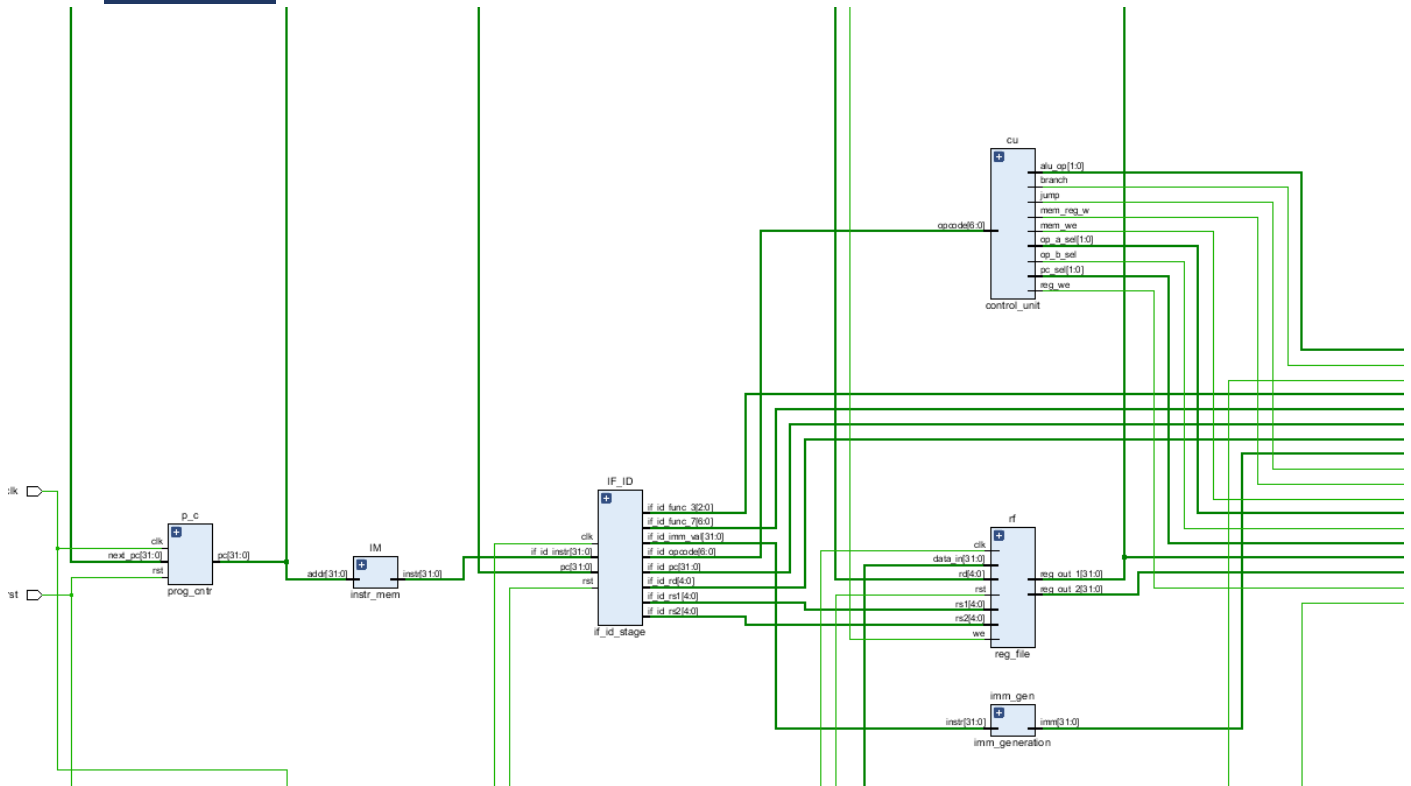
Lab Manual # 21 – Pipelining

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>1st Sep 2025</u></i>

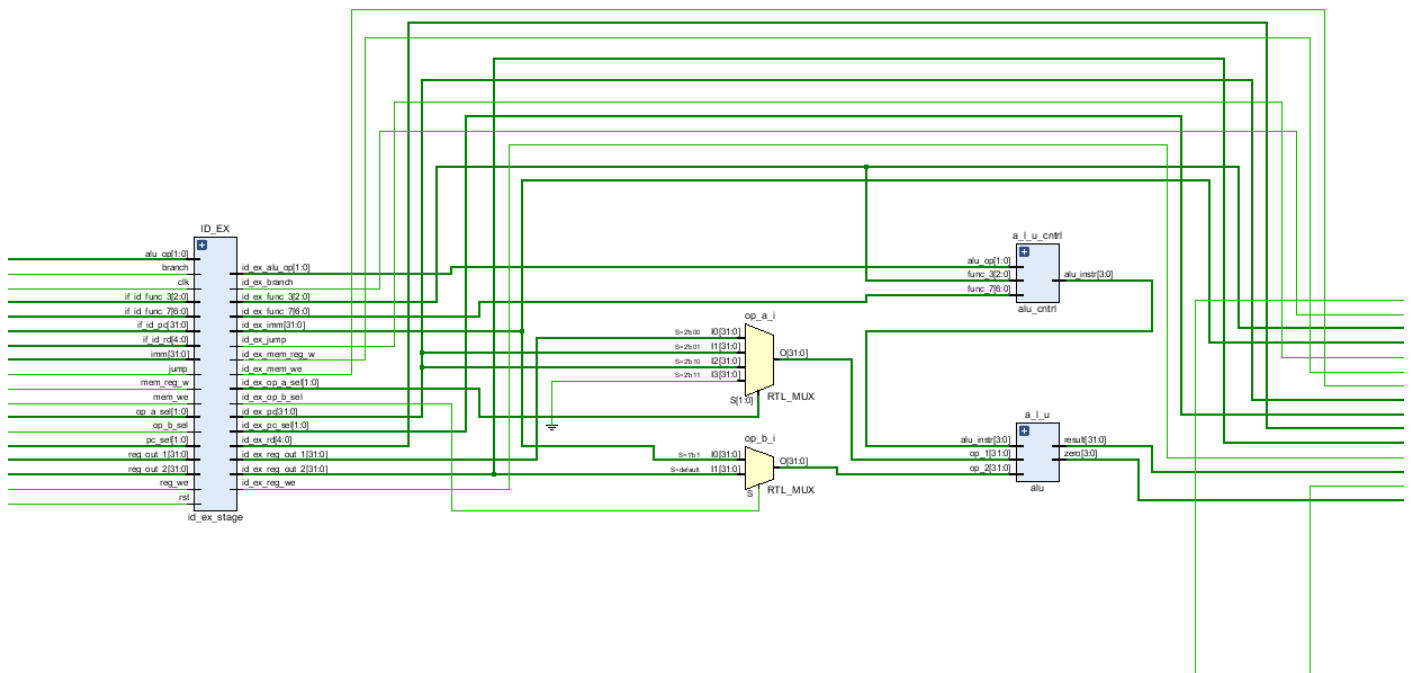
1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Schematics:

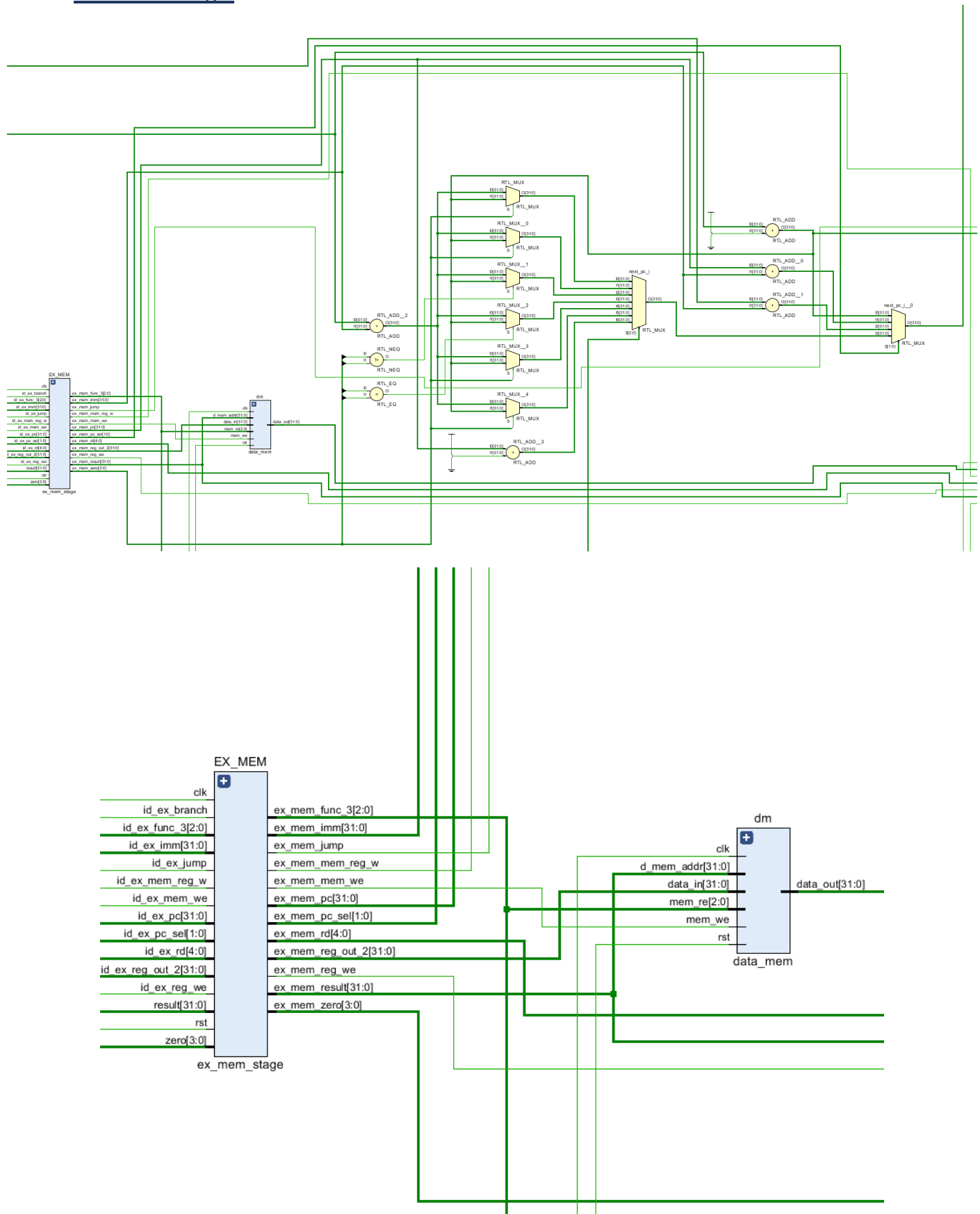
a. IF-ID Stage:



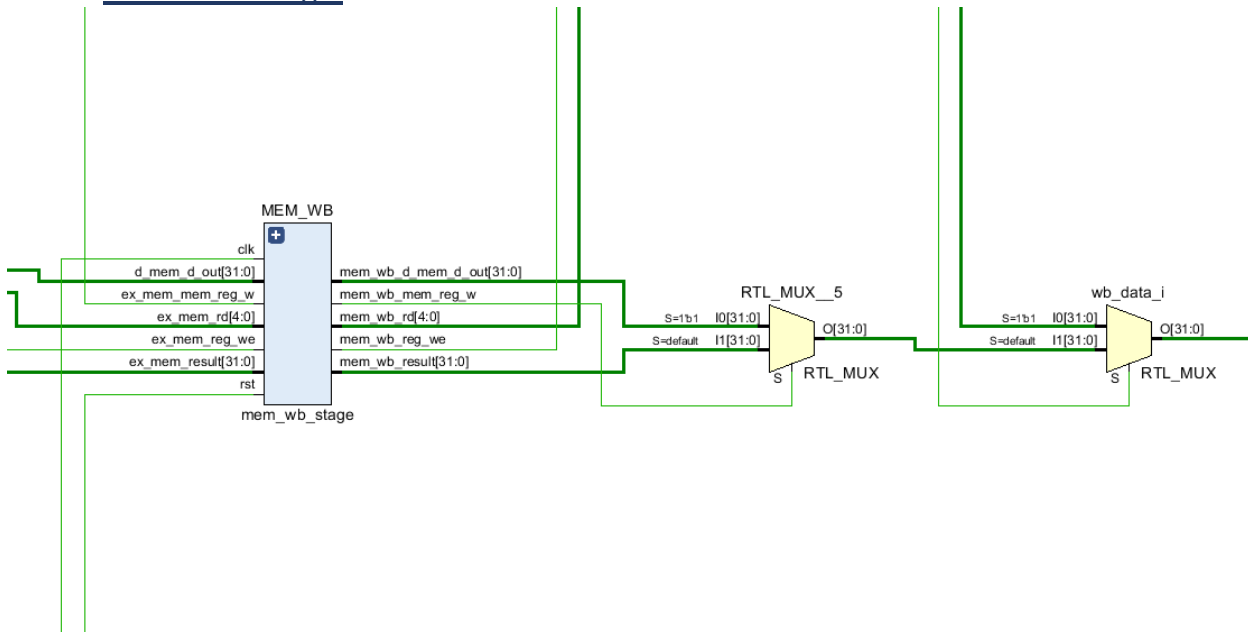
b. ID-EX Stage:



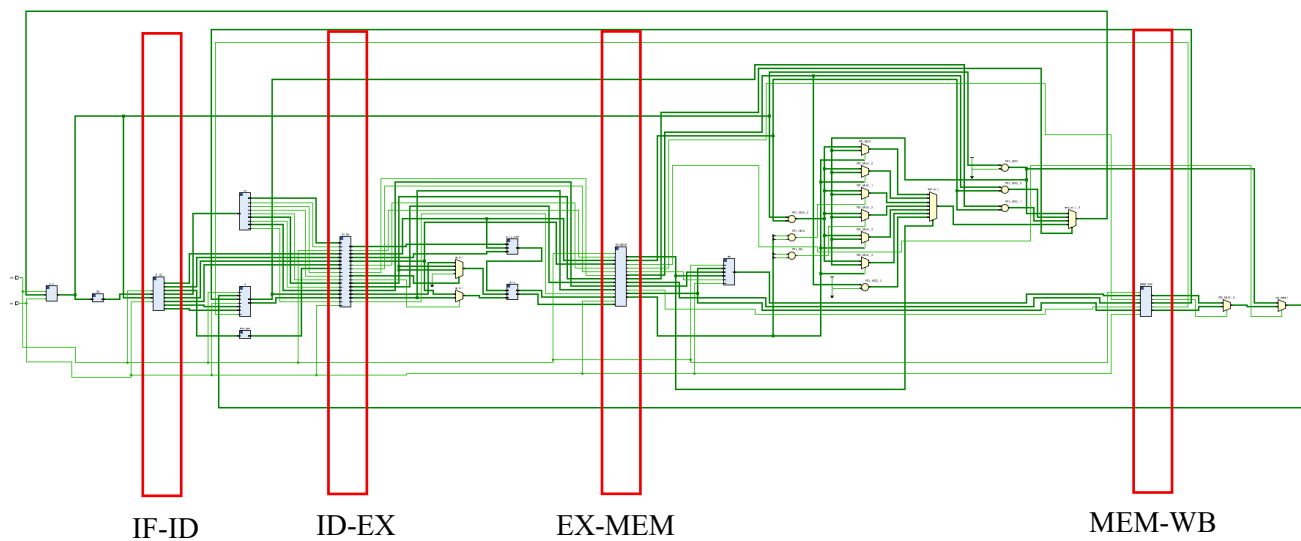
c. EX-MEM Stage:



d. MEM-WB Stage:



e. TOP-5-Stage-Pipeline:



ii. Simulation:

Venus Code:

```
addi x3, x0, 100    # x3 = 100
addi x7, x0, 7      # x7 = 7
nop                # wait for x3, x7 to be ready
nop
nop
nop
nop
sw  x7, 84(x3)      # MEM[100+84] = x7
nop
nop
nop
nop
lw  x5, 84(x3)      # x5 = MEM[100+84]
nop                # wait for lw result
nop
nop
nop
nop
add  x6, x5, x7      # x6 = x5 + x7
addi x10, x0, 10     # x10 = 10
nop
nop
nop
nop
add  x11, x6, x10    # x11 = x6 + x10

# ---- Branch Section ----
addi x12, x0, 17     # x12 = 17
addi x13, x0, 17     # x13 = 17
nop                # let x12, x13 settle
nop
nop
```

```
nop                # let x12, x13 settle
nop
nop
beq  x12, x13, LABEL # branch if x12 == x13
nop                # delay slot (not in RISC-V, but keep
pipeline safe)
nop
nop
nop

# fall-through if branch not taken
addi x14, x0, 99     # x14 = 99
nop
nop
nop
nop
nop
jal  x0, EXIT        # jump to exit

LABEL:
    addi x14, x0, 111 # x14 = 111 (executed if branch
taken)
    nop
    nop
    nop
    nop

EXIT:
    addi x15, x0, 55  # x15 = 55 (program end
marker)
```

Screenshots:

▼ 🚩 register_file[0:31][31:0]	00000000,0000	zero	0x00000000
> 🚩 [0][31:0]	00000000	ra (x1)	0x00000000
> 🚩 [1][31:0]	00000000	sp (x2)	0x00000000
> 🚩 [2][31:0]	00000000	gp (x3)	0x00000064
> 🚩 [3][31:0]	00000064	tp (x4)	0x00000000
> 🚩 [4][31:0]	00000000	t0 (x5)	0x00000007
> 🚩 [5][31:0]	00000007	t1 (x6)	0x0000000E
> 🚩 [6][31:0]	0000000e	t2 (x7)	0x00000007
> 🚩 [7][31:0]	00000007	s0 (x8)	0x00000000
> 🚩 [8][31:0]	00000000	s1 (x9)	0x00000000
> 🚩 [9][31:0]	00000000	a0 (x10)	0x0000000A
> 🚩 [10][31:0]	0000000a	a1 (x11)	0x00000018
> 🚩 [11][31:0]	00000018	a2 (x12)	0x00000011
> 🚩 [12][31:0]	00000011	a3 (x13)	0x00000011
> 🚩 [13][31:0]	00000011	a4 (x14)	0x0000006F
> 🚩 [14][31:0]	0000006f	a5 (x15)	0x00000037
> 🚩 [15][31:0]	00000037		
> 🚩 [16][31:0]	00000000		

These are results comparison. Now the results are incredibly true.

2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, I implemented a **pipelined version of the single-cycle processor** by introducing pipeline registers for all five stages. The registers **IF-ID, ID-EX, EX-MEM, and MEM-WB** were added to transfer instruction and data signals across pipeline stages. To handle hazards at this stage, I inserted **4 NOPs after each instruction** in the assembly program. With this approach, the pipeline executed instructions correctly and produced the expected results. The successful implementation verified the working of pipeline registers in maintaining instruction flow. This lab helped me understand how pipelining improves instruction throughput and prepares the base for hazard detection and forwarding mechanisms in later designs.